Object-Oriented Design
- Design POST

Zhiming Liu
zhimingliu88@swu.edu.cn
Centre for Research and Innovation of Software Engineering
School of Computer and Information Science
Southwest University, Chongqing, China

Design a Project

To design a project,

- We have/need the following models

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts
- We have the deign method:

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts
- We have the deign method: design patterns

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts
- We have the deign method: design patterns
- Then we design each operation of each use case following the patterns

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts
- We have the deign method: design patterns
- Then we design each operation of each use case following the patterns
- Represent the design of an operation by

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts
- We have the deign method: design patterns
- Then we design each operation of each use case following the patterns
- Represent the design of an operation by object sequence diagram

Design a Project

To design a project,

- We have/need the following models
  1. Use case sequence diagram for interactions
  2. Conceptual class diagrams
  3. Identified and named systems operations and their contracts
- We have the deign method: design patterns
- Then we design each operation of each use case following the patterns
- Represent the design of an operation by object sequence diagram
- The object sequence diagram represents the body of the method for the use case operation

Design POST

Design the use case *Process Sale with Cash Payment*

1. Create a separate **object sequence diagram** for **each use case operation** whose **contracts** are defined:

   For each use case op(), make a diagram with it as message to the **controller object**.

Design POST

Design the use case *Process Sale with Cash Payment*

1. Create a separate **object sequence diagram** for **each use case operation** whose **contracts** are defined:

   For each use case op(), make a diagram with it as message to the **controller object**.

2. If the diagram gets complex, split it into smaller diagrams.

Design POST

Design the use case *Process Sale with Cash Payment*

1. Create a separate **object sequence diagram** for **each use case operation** whose **contracts** are defined:

   For each use case op(), make a diagram with it as message to the **controller object**.

2. If the diagram gets complex, split it into smaller diagrams.
3. For some functionality, mostly outputs, we may need to create a separate object sequence diagram, e.g. total()

For each operation or functionality, we need to closely use

Design POST

Design the use case *Process Sale with Cash Payment*

1. Create a separate **object sequence diagram** for **each use case operation** whose **contracts** are defined:

   For each use case op(), make a diagram with it as message to the **controller object**.

2. If the diagram gets complex, split it into smaller diagrams.
3. For some functionality, mostly outputs, we may need to create a separate object sequence diagram, e.g. total()

For each operation or functionality, we need to closely use

- the contract of the operation, conceptual class diagram, and follow GRASP patterns

Design POST

Design the use case *Process Sale with Cash Payment*

1. Create a separate **object sequence diagram** for **each use case operation** whose **contracts** are defined:

   For each use case op(), make a diagram with it as message to the **controller object**.

2. If the diagram gets complex, split it into smaller diagrams.
3. For some functionality, mostly outputs, we may need to create a separate object sequence diagram, e.g. total()

For each operation or functionality, we need to closely use

- the contract of the operation, conceptual class diagram, and follow GRASP patterns
- A responsibility assigned to a class is also a method of the class

Design POST

Design the use case *Process Sale with Cash Payment*

1. Create a separate **object sequence diagram** for **each use case operation** whose **contracts** are defined:

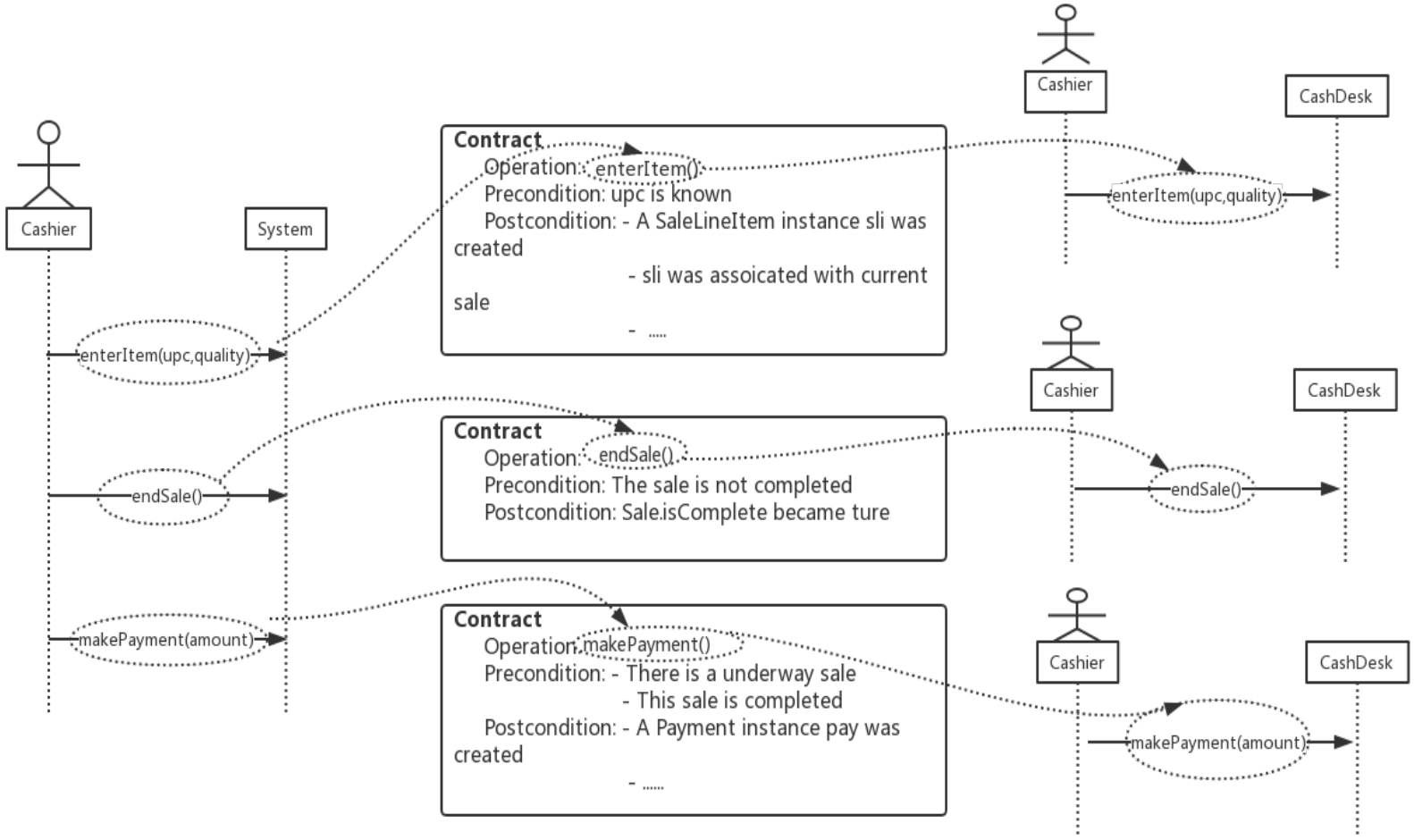
   For each use case op(), make a diagram with it as message to the **controller object**.

2. If the diagram gets complex, split it into smaller diagrams.
3. For some functionality, mostly outputs, we may need to create a separate object sequence diagram, e.g. total()

For each operation or functionality, we need to closely use

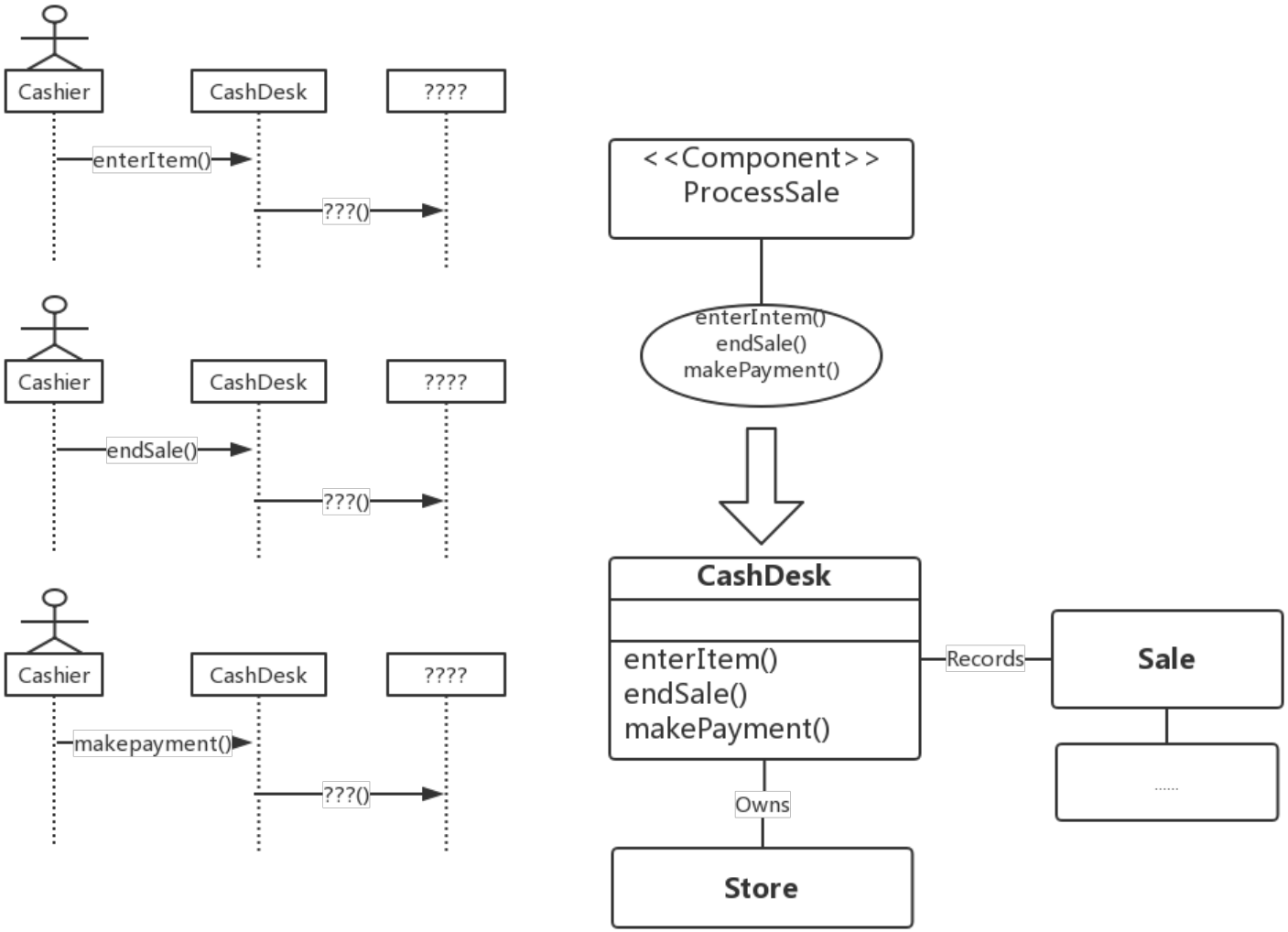
- the contract of the operation, conceptual class diagram, and follow GRASP patterns
- A responsibility assigned to a class is also a method of the class
- The conceptual class diagram is thus refined to a design class diagram
- For each use case, we use an **interface class** as the controller, following the Controller Pattern

Use Case Design Overview



Thanks to Yukun Zhang for the diagram

Overview of Use Case Operation Design



Thanks to Yukun Zhang for the diagram

Design *enterItem*()

Recall its contract

Responsibilities: Record sale of an item, display item description and price.

Pre-conditions: The upc is known to the system.

Post-conditions:

- if a new sale, a *Sale* was created.
- if a new sale, the new *Sale* was associated with the *POST*.
- a *SalesLineItem* was created.
- *SalesLineItem.quantity* was set to *quantity*.
- *SalesLineItem* was associated with a *ProductSpecification*, based on *UPC* match.
- *SalesLineItem* is added to *Sale*

Displaying Item Description and Price

- Model objects are not responsible for display of information
- These are the responsibilities of GUI objects that
  - access to data known by model objects, and
  - can send messages to model objects.

Displaying Item Description and Price

- Model objects are not responsible for display of information
- These are the responsibilities of GUI objects that
  - access to data known by model objects, and
  - can send messages to model objects.
- However, model objects do not send messages to interface objects in order to keep the system independent of the interface.
- This is a matter of interface design and we leave it for later discussion.

- Decide the controller object for *enterItem*(), apply Controller Pattern

- Decide the controller object for *enterItem*(), apply Controller Pattern

  CashDesk

- Decide the controller object for *enterItem*(), apply Controller Pattern

  CashDesk

Design methods for postconditions

The post conditions in the contract of *enterItem* include:

- If a new sale, a *Sale* was created.
- If a new sale, the new *Sale* was associated with the *CashDesk*.

Follow Creator Pattern

Design methods for postconditions

The post conditions in the contract of *enterItem* include:

- If a new sale, a *Sale* was created.
- If a new sale, the new *Sale* was associated with the *CashDesk*.

Follow Creator Pattern

Also create an empty container class $<<$ *set* $>>$ *SalesLineItem*

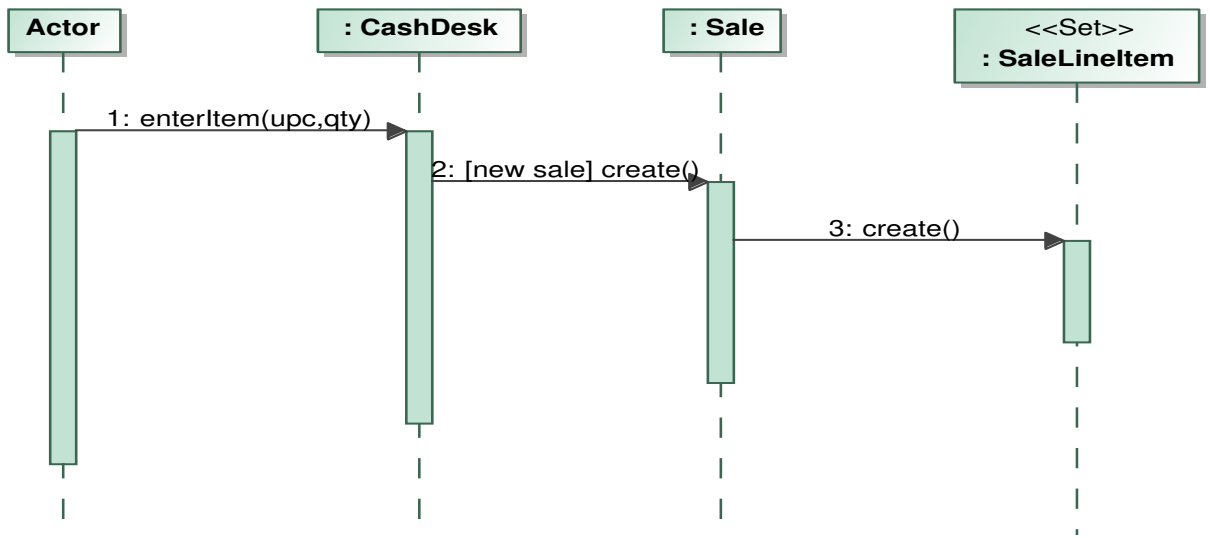
Design methods for postconditions

The post conditions in the contract of *enterItem* include:

- If a new sale, a *Sale* was created.
- If a new sale, the new *Sale* was associated with the *CashDesk*.

Follow Creator Pattern

Also create an empty container class $<<$ *set* $>>$ *SalesLineItem*



But, is the new Sale associated with *CashDesk*?

Creating a New *SalesLineItem*

Other post-conditions in the contract for *enterItem*() include:

- A *SalesLineItem* was created.
- The *SalesLineItem* was associated with the *Sale*.
- *SalesLineItem.quantity* was set to *qty*.
- The *SalesLineItem* was associated with the *ProductSpecification*, based on upc match.

The first indicates the responsibility to create a *SalesLineItem* instance

Creating a New *SalesLineItem*

Other post-conditions in the contract for *enterItem*() include:

- A *SalesLineItem* was created.
- The *SalesLineItem* was associated with the *Sale*.
- *SalesLineItem.quantity* was set to *qty*.
- The *SalesLineItem* was associated with the *ProductSpecification*, based on upc match.

The first indicates the responsibility to create a *SalesLineItem* instance

Who should be creator?

By Creator Pattern, the answer is *Sale*

By Creator Pattern, the answer is *Sale*

- SalesLineItem has attributes “quantity”, and ”specification” (at least name of item)

By Creator Pattern, the answer is *Sale*

- SalesLineItem has attributes “quantity”, and ”specification” (at least name of item)
- This implies that the creation of *SalesLineItem* needs input values for these attributes

By Creator Pattern, the answer is *Sale*

- SalesLineItem has attributes “quantity”, and ”specification” (at least name of item)
- This implies that the creation of *SalesLineItem* needs input values for these attributes
- Thus the message *makeLineItem*() from *CashDesk* to the *Sale* should include ”quantity” and *ProductSpecification*, and
- Sale then passes them in the creation message to the *SalesLineItem*.

Finding a *ProductSpecification*

- where to get *ProductSpectification before* *makeItem*(*spec*, *qty*) is sent to the *Sale*?

Finding a *ProductSpecification*

- where to get *ProductSpectification before* *makeItem*(*spec*, *qty*) is sent to the *Sale*?

  who should be responsible for knowing ProductSpecification?

Finding a *ProductSpecification*

- where to get *ProductSpectification before* *makeItem*(*spec*, *qty*) is sent to the *Sale*?

  who should be responsible for knowing ProductSpecification?

- Notice that the postconditions of *enterItem* also requires that

  the newly created SalesLineItem to be associated with the ProductSpecification based on upc match

Finding a *ProductSpecification*

- where to get *ProductSpectification before* *makeItem*(*spec*, *qty*) is sent to the *Sale*?

  who should be responsible for knowing ProductSpecification?

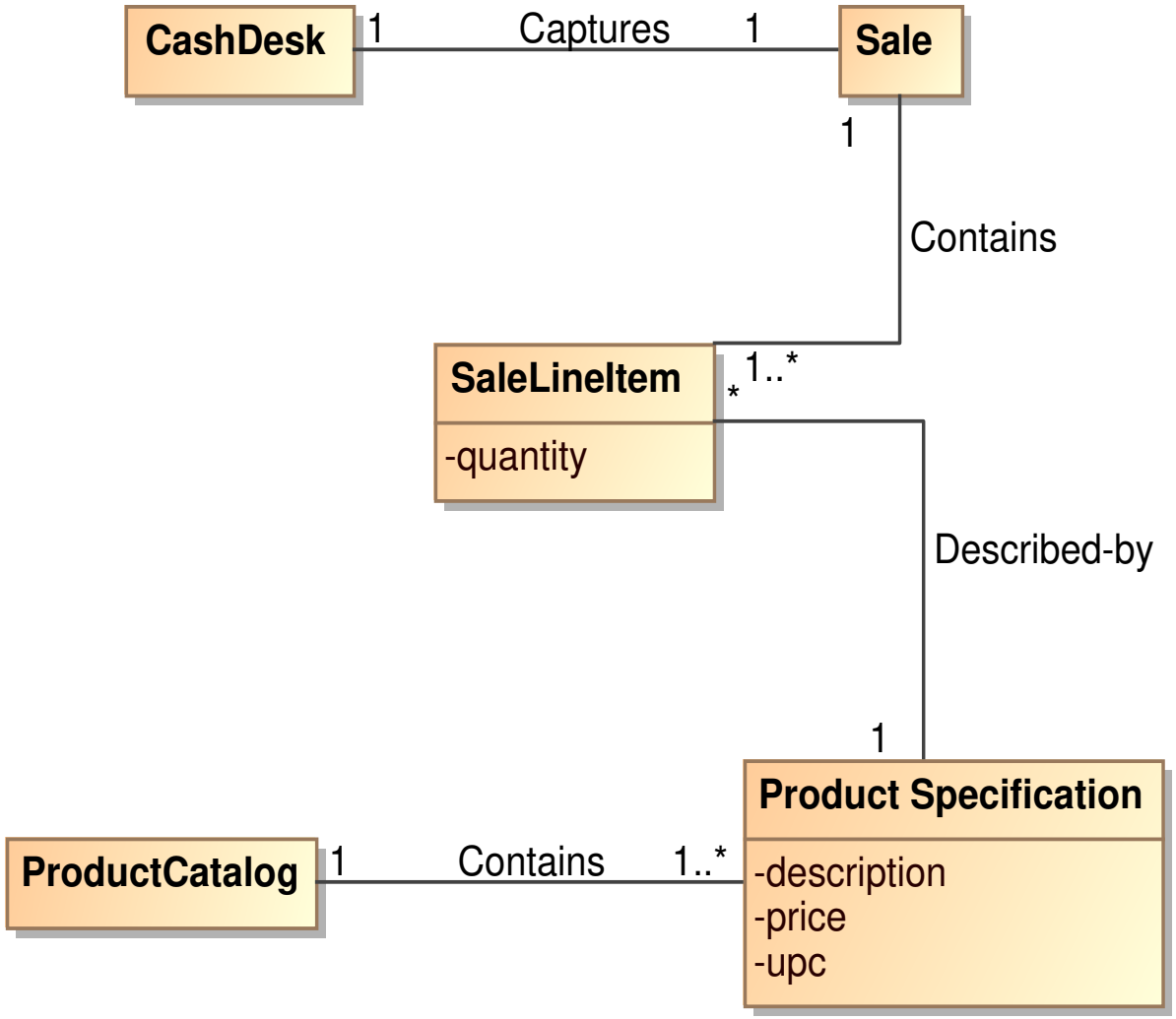
- Notice that the postconditions of *enterItem* also requires that

  the newly created SalesLineItem to be associated with the ProductSpecification based on upc match

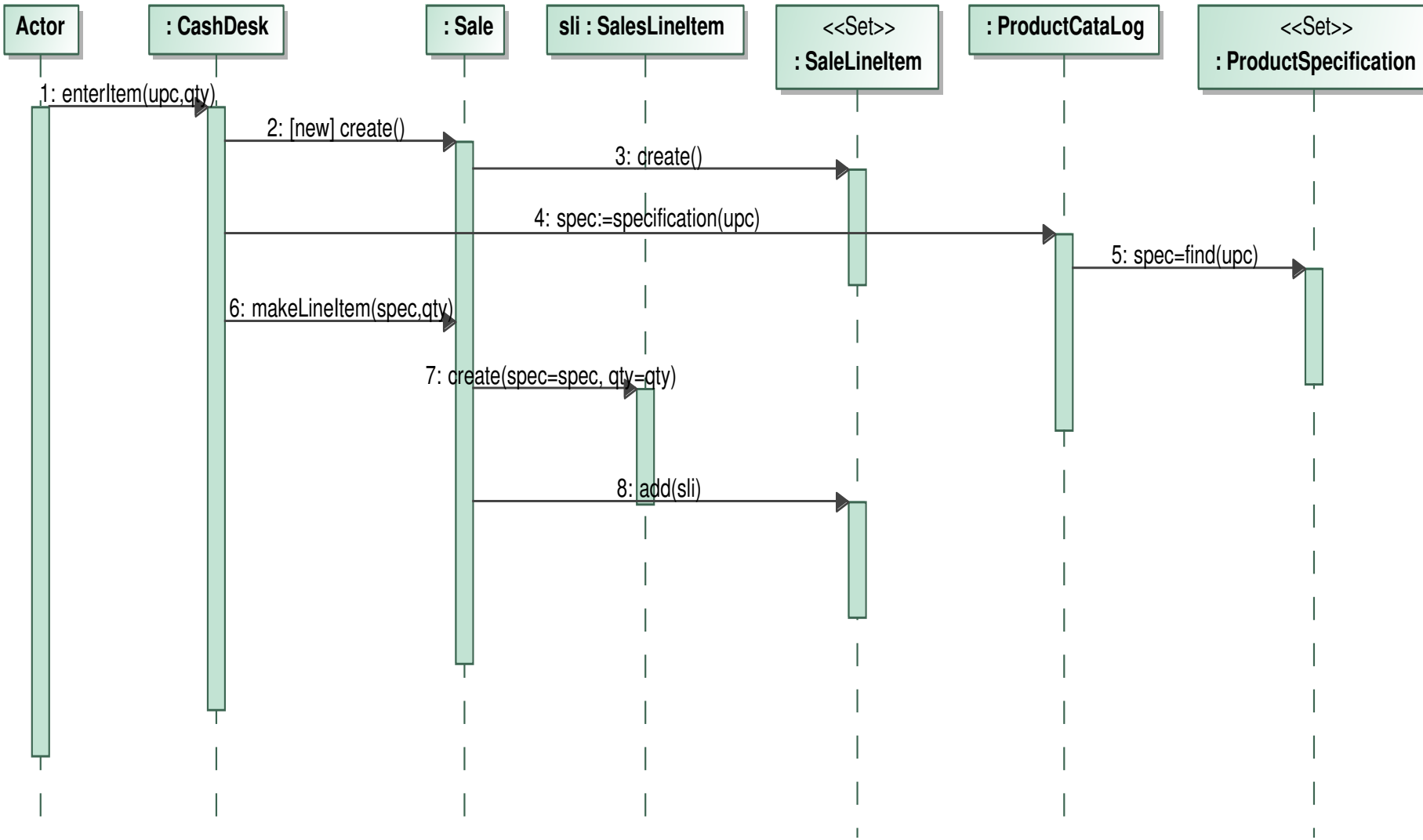
- Thus, the Answer is that *ProductCatalog* should be the expert knowing and looking up a *ProductSpecification*, based on UPC match.

Finally after the SalesLineItem is created, it is associated with the sale, **but how?**

Part of the Conceptual Model

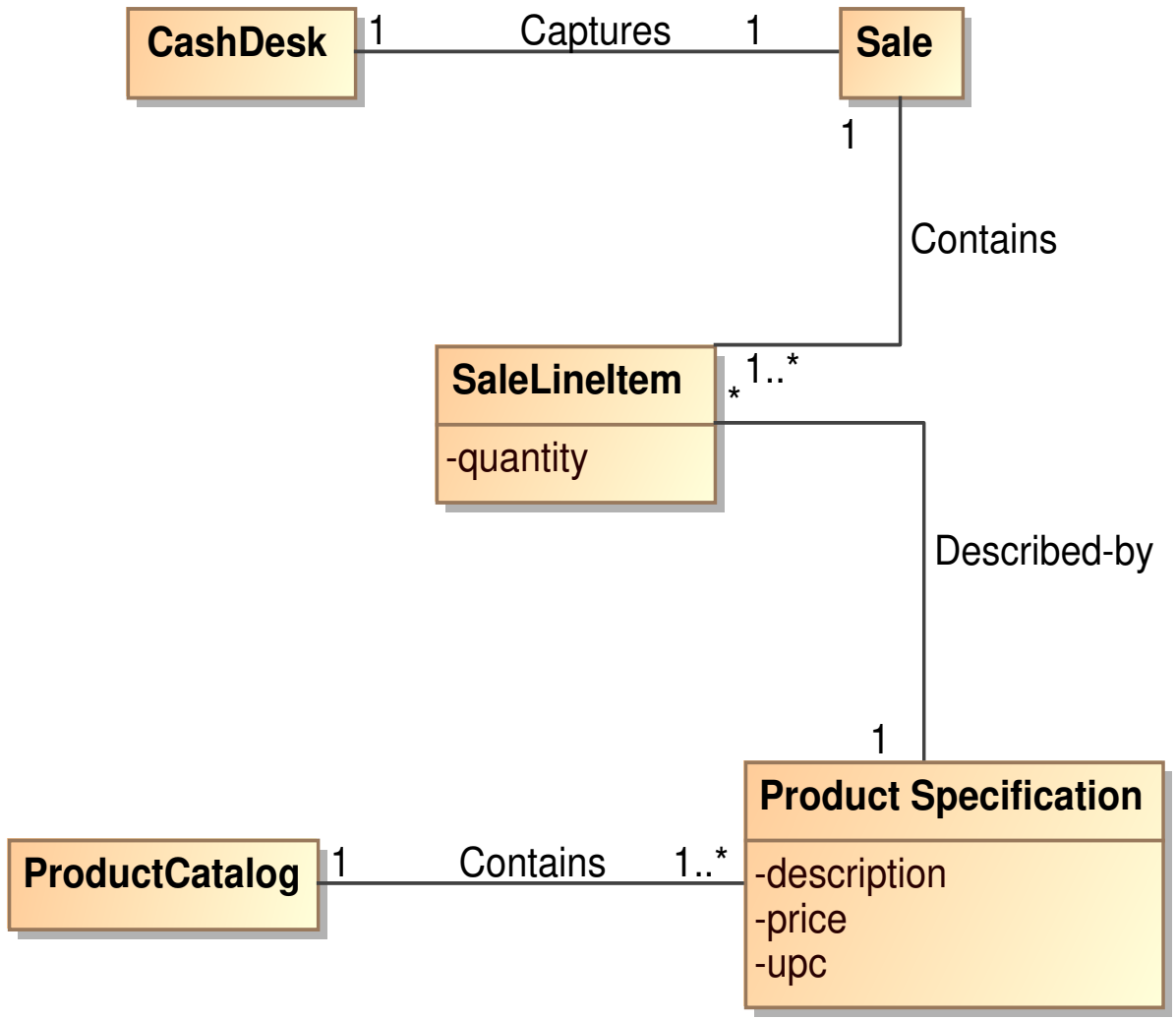


Complete Design of *enterItem(upc, qty)*



Check the Class Diagram Again

Anything missing from the class diagram?



Pseudo-code for *enterItem*(*upc*, *qty*)

CashDesk:: enterItem(upc,qty) if new sale then
<< *new* >>Sale.create()
spec:=ProductCataLog.specification(upc)
Sale.makeLineItem(spec,qty)

NB:

- OOD does do top-down functional decomposition
- also, the decomposition is more systematic
- **reason:** decomposition of the state space has already been largely done when building the class diagram.

Discussion

1. Alternative design?

Discussion

1. Alternative design?
   - create a : *LineItem* without setting the attributes
   - then get the *spec* and *price*
   - finally sets the attributes *spec* and *price*
2. Validation/justification of design
   - check each postcondition is met in the object sequence diagram

Design *finishSale*()

Recall the contract

Responsibilities: calculate *total*() and the postcondition

Name: endsale().

Responsibilities: Record that it is the end of entry of sale items, and display sale total.

Pre-conditions:

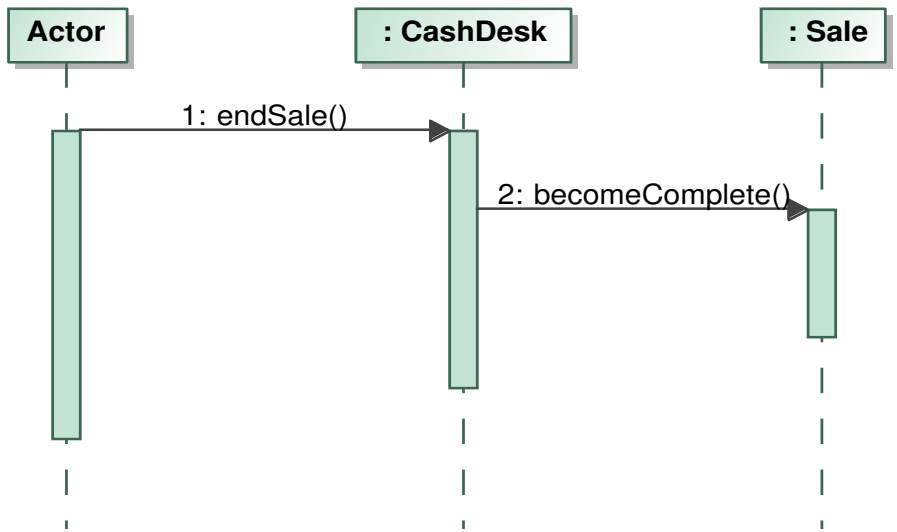
Post-conditions: *Sale.isComplete* was set to *true* (attribute modification).

Set *Sale.isComplete*

who should be responsible for setting the *isComplete* attribute of the *Sale* to true?

Set *Sale.isComplete*

who should be responsible for setting the *isComplete* attribute of the *Sale* to true?

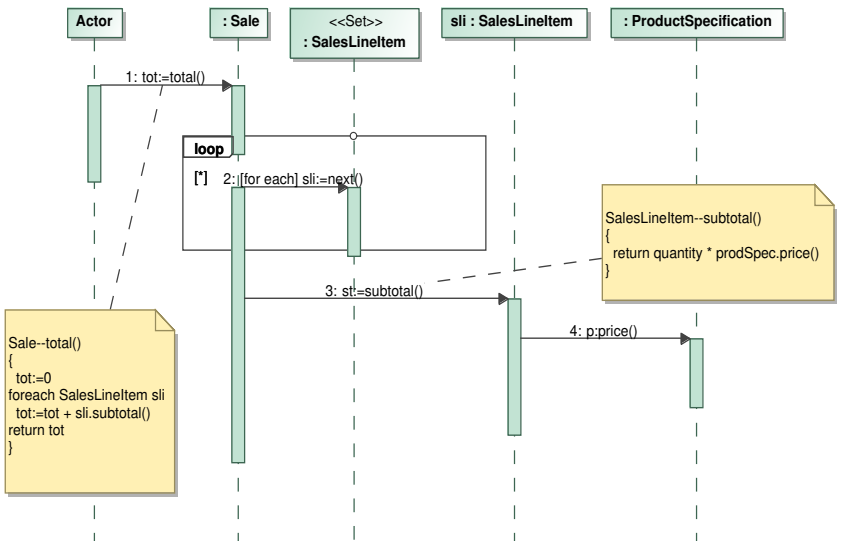


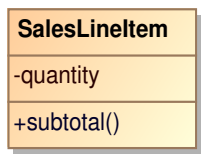
Calculating the *Sale* total

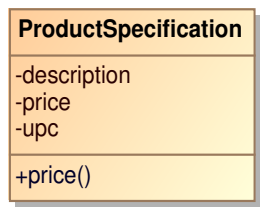
- not design GUI to display the total
- but, design the calculation to be displayed

Calculating the *Sale* total

- not design GUI to display the total
- but, design the calculation to be displayed

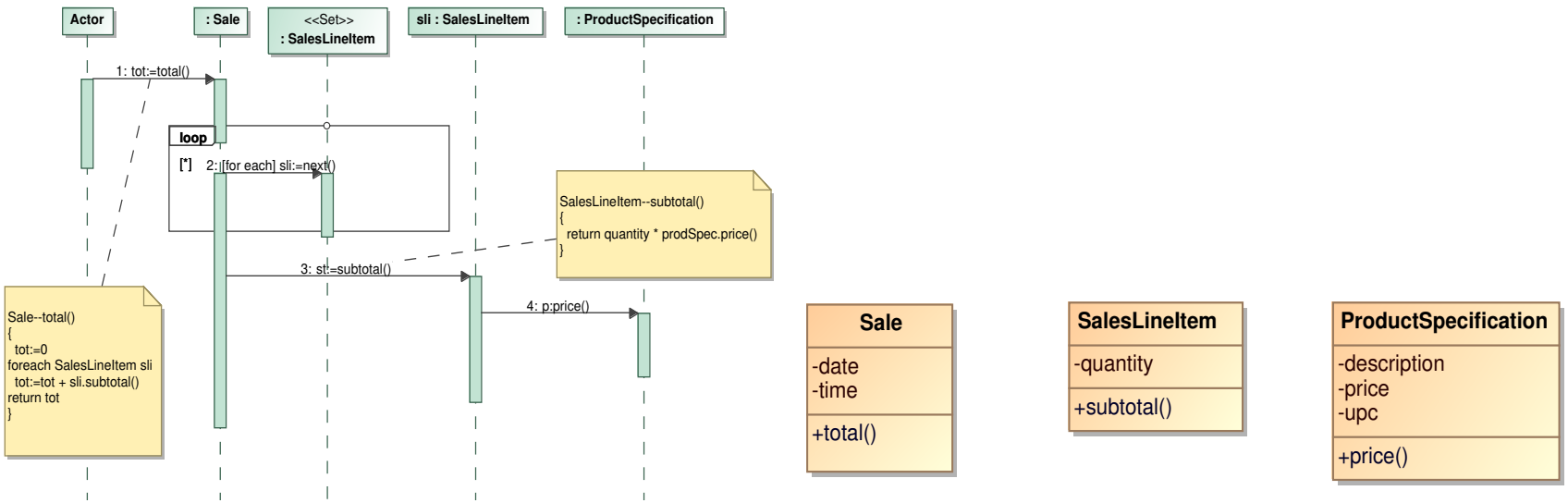






Calculating the *Sale* total

- not design GUI to display the total
- but, design the calculation to be displayed



Who will send the *total* message to the *Sale*?

Design *cashPay()*

Contract

Name: makePayment(amount: Quantity).

Responsibilities: Record the payment, calculate balance and print receipt.

Post-conditions:

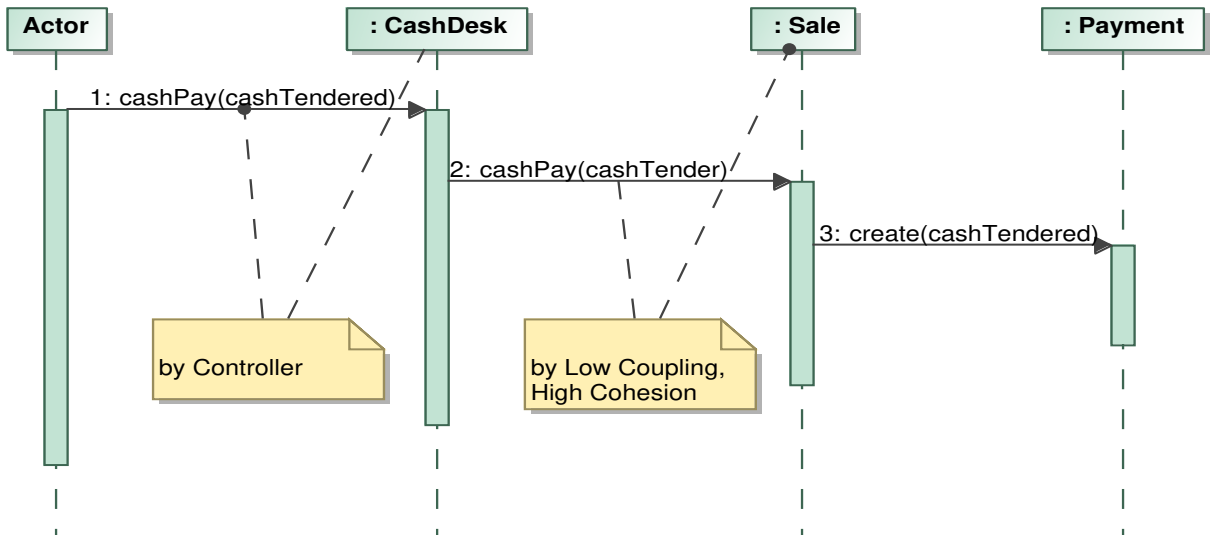
- A *Payment* was created (instance creation).
- *Payment.amountTendered* was set to *amount*.
- The *Payment* was associated with the *Sale*.
- The *Sale* was associated with the *Store*, to add it to the historical log of completed sales.

Design makePayment()

- We gave a design, what was it?

Design makePayment()

- We gave a design, what was it?
- By Creator, Low coupling and high cohesion, *Sale* is the creator of *Payment*

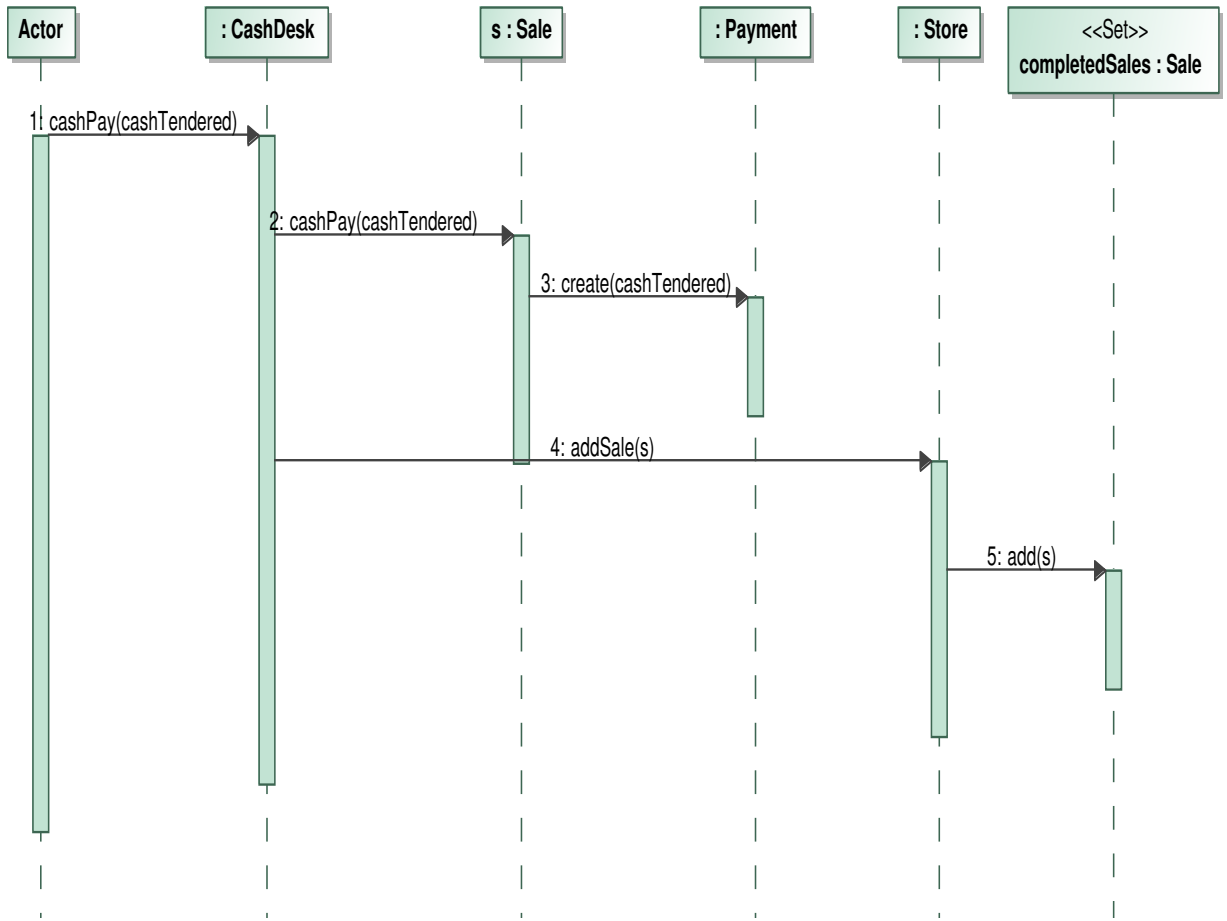


Logging the *Sale*

By **Expert**

Logging the *Sale*

By **Expert**

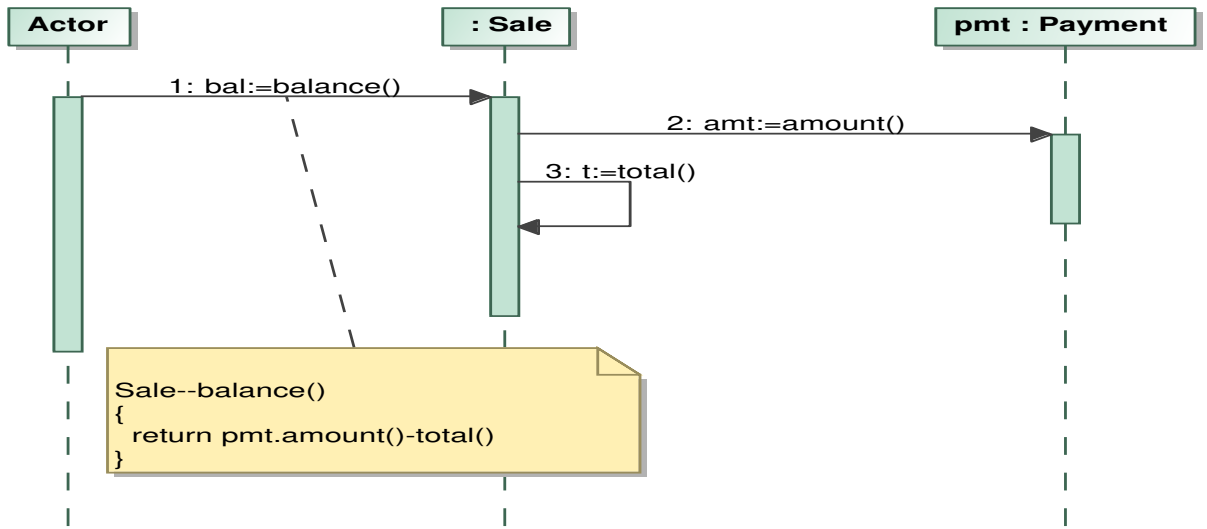


Calculating Balance

- by **Expert**, either *Sale* or *Payment*
- by **Low coupling**, *Sale* is better.

Calculating Balance

- by **Expert**, either *Sale* or *Payment*
- by **Low coupling**, *Sale* is better.



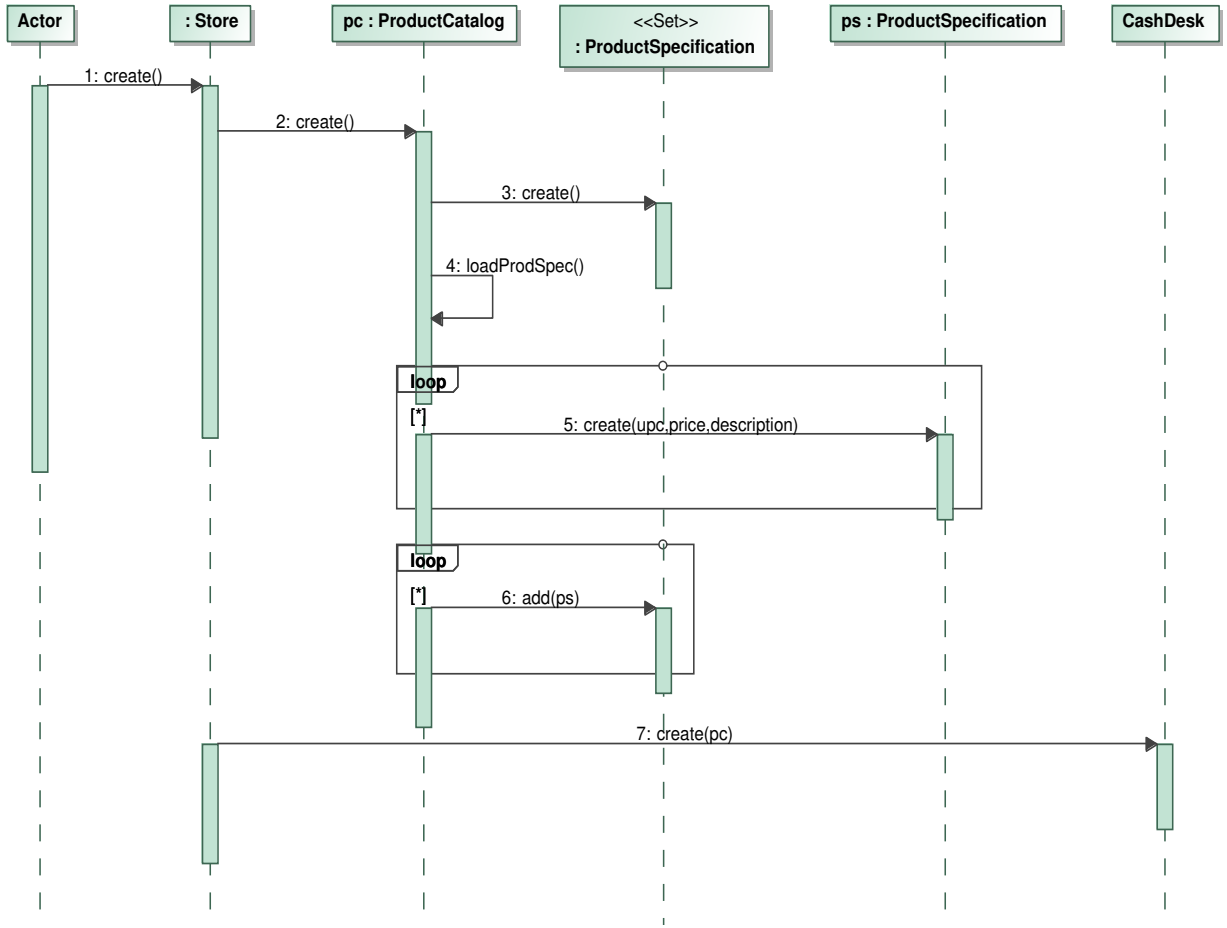
StartUp

- all components must be initialized by a *StarUp*() operation before any other operations can be used
- better to delay the design of *StartUp*() until after the other interface methods are designed

Post Conditions of *startUp*

- A Sore, CashDesk, ProductCatalog and ProductSpecification were created (instance creation).
- *ProductCatalog* was associated with *ProductSpecification* (association formed).
- *Store* was associated with *ProductCatalog* (association formed).
- *Store* was associated with *CashDesk* (association formed).
- *CashDesk* was associated with *ProductCatalog* (association formed)

Design *StartUp*()



Conclusions on OO Design

- OO design = creation of interaction diagrams for each system operations
  - Based on the contracts of the operations
  - Follow design patterns
  - Also refine the conceptual class diagram to the design class diagram

Conclusions on OO Design

- OO design = creation of interaction diagrams for each system operations
  - Based on the contracts of the operations
  - Follow design patterns
  - Also refine the conceptual class diagram to the design class diagram
- Validation of the design against the contracts and conceptual class diagram

Conclusions on OO Design

- OO design = creation of interaction diagrams for each system operations
  - Based on the contracts of the operations
  - Follow design patterns
  - Also refine the conceptual class diagram to the design class diagram
- Validation of the design against the contracts and conceptual class diagram
- OO implementation = translation diagrams into code – easy.
- OO design is the most technical and demands a lot of skills
  - Algorithm design and design patterns

Conclusions on OO Design

- OO design = creation of interaction diagrams for each system operations
  - Based on the contracts of the operations
  - Follow design patterns
  - Also refine the conceptual class diagram to the design class diagram
- Validation of the design against the contracts and conceptual class diagram
- OO implementation = translation diagrams into code – easy.
- OO design is the most technical and demands a lot of skills
  - Algorithm design and design patterns
- incremental and iterative design is supported

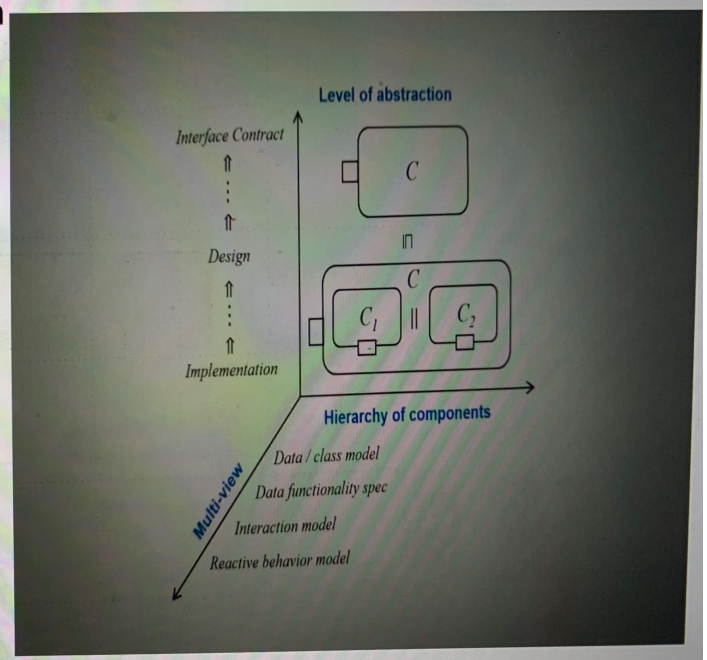
Need to practice through a whole career time

Use Case Driven Component-Based Development

Model-Driven for Predictability and Dependability

Build ***system models*** to gain confidence in requirements and designs

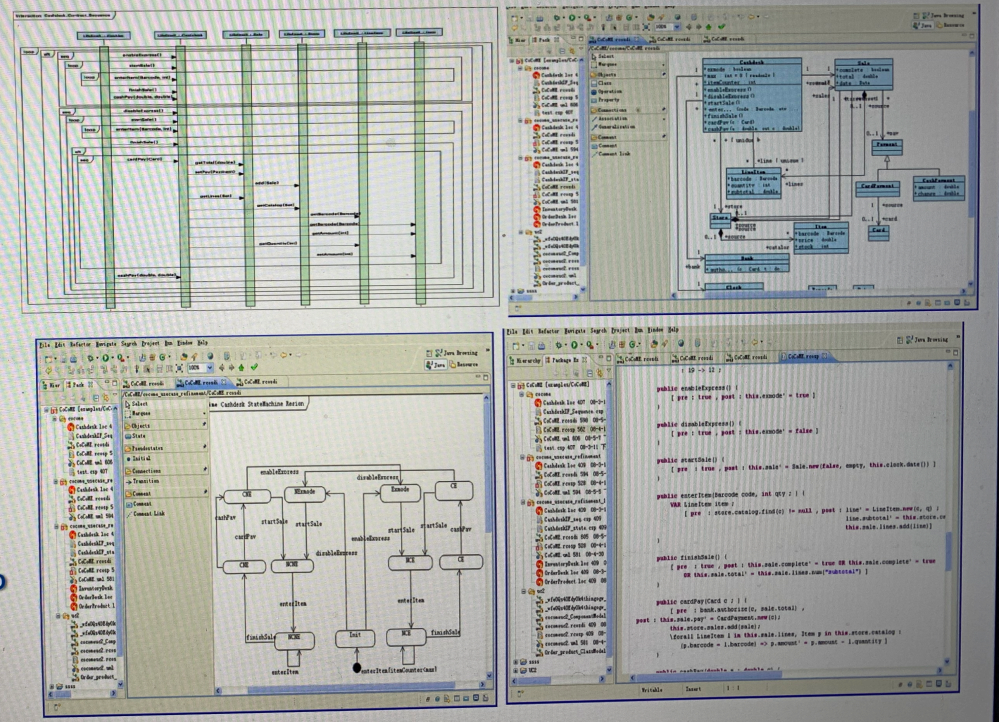
- Use **abstraction** for information hiding
  - well-focused
  - problem-oriented
- Use **decomposition** and **separation of concerns**
  - divide and conquer
  - incremental development
- use **rigor/formalization**
  - repeatable process
  - analyzable artifacts
- rCOS is fully formalizable, but allows various degree of formability in applications
- Basis for Tool Support



Use Case Driven Component-Based Development

Model–Driven Development with rCOS

- Each phase is based on the construction of ***verifiable models***
- Models are analyzed and verified
- Refined models are constructed by ***model transformations***
- ***Code*** is generated from design models
- ***Proof obligations*** are generated by model transformations
- ***rCOS modeler*** integrates UML model notation into rCOS



Model Transformations in rCOS